DOMAIN DRIVEN DESIGN USING JAVA

SpotMyDeal

Entrega da 4ª Sprint — API Restful com Spring Boot e Oracle

NOME DA EQUIPE

NextStage

NOME DA SOLUÇÃO

SpotMyDeal

INTEGRANTES

Tayna Jimenes Silva	RM 569337
Gabriel Augusto	RM 573120
Rodrigo Banharelli de Souza	RM 570539
Nycolas Melo Escobar	RM 573052

Sumário

1 Objetivo 3

2 Escopo 3

3 Funcionalidades implementadas 4

4 Arquitetura em camadas 5

5 Modelo de Entidade-Relacionamento 6

6 Diagrama de classes 7

7 Endpoints da API 9

8 Regras de negócio 10

9 Tratamento de erros 11

10 Deploy em nuvem 12

11 Integração com o front-end 13

12 Protótipo — telas do sistema 14

13 Procedimentos para execução 17

14 Repositórios e links 19

1 Objetivo

Desenvolver uma plataforma Java que usa gamificação para incentivar práticas sustentáveis. O sistema conecta usuários a missões ambientais e converte ações positivas em benefícios reais, na forma de cupons em lojas parceiras próximas a quem cumpriu a missão.

Nesta quarta sprint, a solução deixou de ser uma aplicação de console e passou a ser uma **API Restful**, publicada em nuvem e consumida por uma interface web. O domínio continua o mesmo das sprints anteriores; o que mudou foi a forma de acesso e a organização interna do código.

2 Escopo

Perfil do usuário

Cadastro, consulta, alteração e remoção de usuários, com progresso, pontuação acumulada e nível de engajamento calculado a partir dos pontos.

Catálogo de missões

Missões sustentáveis com regra de recompensa e prazo. Cada missão vale uma quantidade de pontos e tem um limite de dias para ser considerada válida.

Geolocalização

Validação de proximidade entre o usuário e as lojas parceiras, com cálculo de distância real sobre coordenadas geográficas e listagem das lojas dentro de um raio informado.

Sistema de gamificação

Registro de conclusão de missão, progressão automática de nível, ranking dos usuários por pontuação e liberação condicionada de benefícios.

Validação de ofertas

Controle do resgate de cupom com base em dois critérios simultâneos: a localização física do usuário no momento do resgate e o histórico recente de atividades sustentáveis.

> **Correção em relação à 3ª Sprint.** O modelo anterior guardava a conclusão da missão como um atributo da própria missão. O efeito era que, quando um usuário concluía uma missão, ela aparecia concluída para todos. Na 4ª Sprint isso virou tabela associativa: a conclusão pertence ao par usuário × missão, não à missão sozinha.

3 Funcionalidades implementadas

Tudo o que segue está implementado e respondendo na aplicação publicada. Não há recurso descrito aqui que exista apenas no papel.

Cadastro e gestão de usuários

CRUD completo. O cadastro recusa e-mail repetido e nome fora da faixa de 3 a 80 caracteres. A alteração não aceita pontuação nem nível no corpo da requisição: esses campos pertencem ao sistema, e não a quem chama. A remoção é bloqueada quando existe histórico.

Catálogo de missões

CRUD completo, com pontos de recompensa e prazo em dias. Cada usuário tem ainda a consulta das missões que *ainda não* concluiu, que é o que a tela de missões precisa mostrar.

Cadastro de lojas parceiras

CRUD completo, com coordenadas geográficas, código do cupom e indicador de loja ativa. A listagem aceita filtro por lojas ativas.

Conclusão de missão e pontuação

O registro da conclusão soma os pontos ao usuário e promove o nível automaticamente, na mesma operação. A conclusão em duplicidade é recusada.

Busca por proximidade

Dadas uma coordenada e um raio em metros, a API devolve as lojas ativas dentro do raio, ordenadas da mais próxima para a mais distante. A distância é calculada pela fórmula de Haversine, escrita no projeto.

Resgate de cupom com dupla validação

O resgate confere, ao mesmo tempo, a distância até a loja e a existência de missão concluída nos últimos sete dias. É a funcionalidade que amarra geolocalização e gamificação.

Ranking

Lista dos usuários ordenada por pontuação, com a ordenação implementada por Bubble Sort.

Histórico

Consulta das missões concluídas e dos cupons resgatados por usuário, cada uma com data e pontos envolvidos.

4 Arquitetura em camadas

A requisição percorre sempre o mesmo caminho, e sempre num único sentido. Nenhuma camada pula a seguinte, e nenhuma volta para a anterior.



CAMADA	PACOTE	RESPONSABILIDADE
Model	`model`	As cinco entidades do MER, cada uma com os métodos de negócio que dizem respeito a ela mesma
DAO	`dao`	O SQL de uma tabela cada, com `PreparedStatement` e `try-with-resources`
Conexão	`factory`	`ConnectionFactory` sobre o `DataSource` administrado pelo Spring
BO	`bo`	Regras de negócio e validações; orquestra mais de um DAO quando a regra exige
Resource	`resource`	Os endpoints HTTP; recebe e devolve DTO, e não conhece SQL
DTO	`dto`	Contrato de entrada, com Bean Validation, e contrato de saída
Exceções	`exception`	Exceções de negócio e o `@RestControllerAdvice` que as traduz em status HTTP

Por que a BO existe

A camada de regra de negócio é o que separa o que o banco aceita do que o sistema permite. O Oracle barra a exclusão de um usuário com histórico por causa da chave estrangeira, mas o erro que chega ao cliente é uma falha de integridade, genérica e sem explicação. Com a BO no caminho, a mesma situação vira uma resposta `400` dizendo exatamente o que impede a operação e o que fazer a respeito.

Por que o DTO existe

A entidade e o contrato HTTP mudam por motivos diferentes. Separar os dois evita que o JSON exposto dependa do formato das colunas, e permite recusar na entrada campos que o cliente não deve informar — a pontuação, por exemplo, que só muda por conclusão de missão.

Algoritmos escritos à mão

Três cálculos foram implementados sem recorrer a biblioteca pronta: a distância entre duas coordenadas pela fórmula de Haversine, a ordenação do ranking por Bubble Sort e a validação de formato de e-mail por varredura de caracteres.

5 Modelo de Entidade-Relacionamento

Cinco tabelas, todas com o prefixo `T_SMD_`. As duas em verde são associativas e resolvem relações N:N; as chaves primárias são geradas por *sequence*.

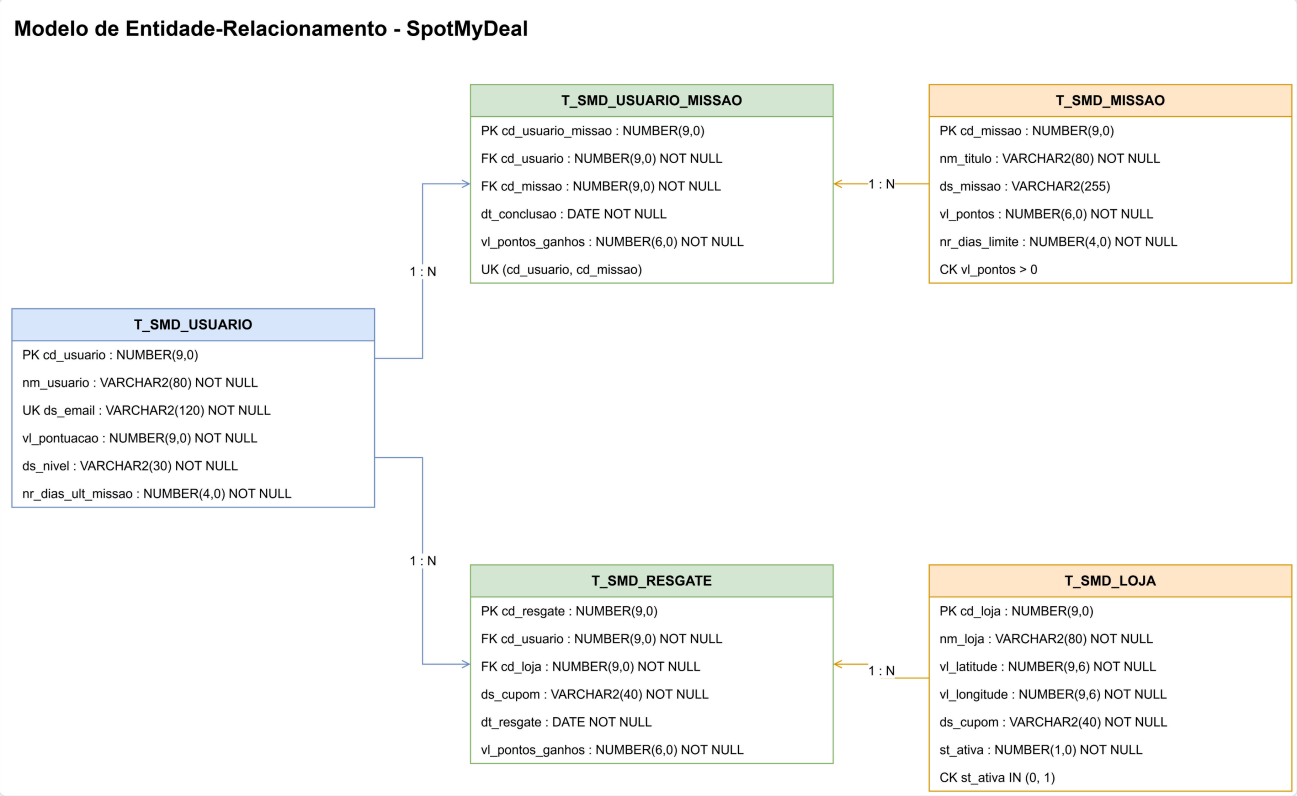


TABELA	O QUE GUARDA
`T_SMD_USUARIO`	Quem participa, com pontuação, nível e dias desde a última missão
`T_SMD_LOJA`	Loja parceira, com coordenadas, cupom e indicador de loja ativa
`T_SMD_MISSAO`	Missão sustentável, com pontos de recompensa e prazo em dias
`T_SMD_USUARIO_MISSAO`	A conclusão: qual usuário concluiu qual missão, quando e por quantos pontos
`T_SMD_RESGATE`	O resgate: qual usuário pegou qual cupom, em qual loja e quando

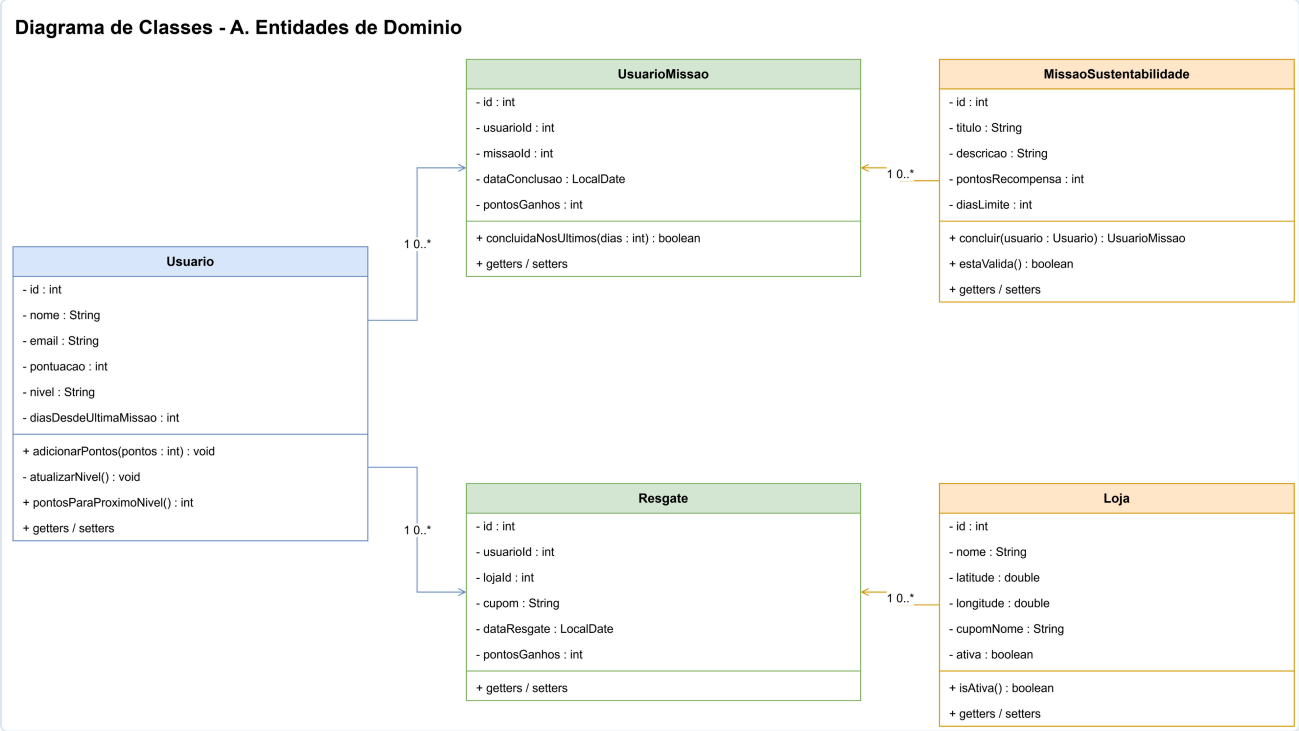
A chave única sobre `(cd_usuario, cd_missao)` é o que garante, no próprio banco, que ninguém conclui a mesma missão duas vezes.

6 Diagrama de classes

A estrutura foi dividida em duas visões, para que cada uma caiba numa leitura.

A. Entidades de domínio

As classes que representam os dados e carregam as regras de negócio de cada entidade.



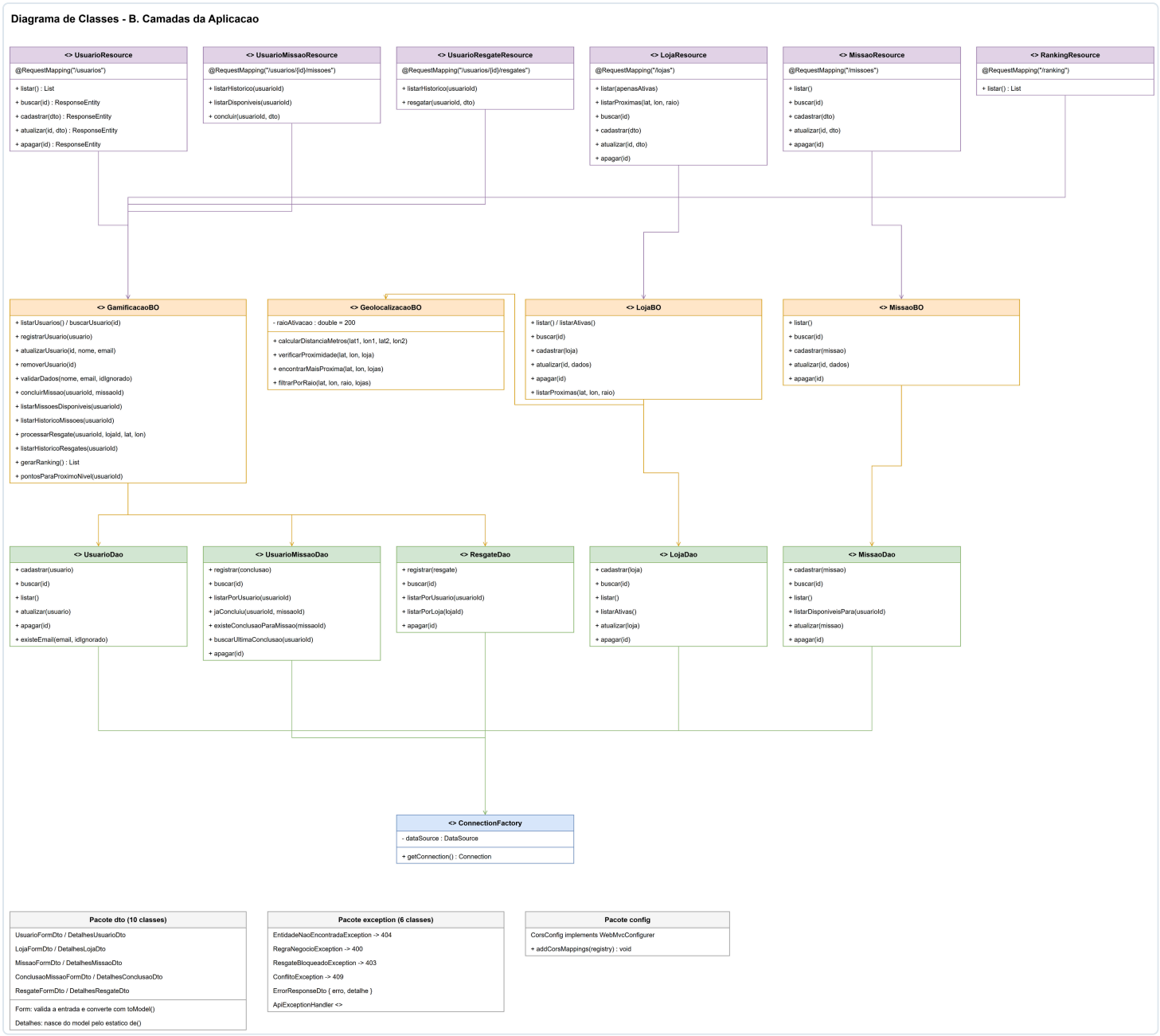
Métodos de negócio de cada entidade

Além dos construtores e acessadores, cada classe carrega os cálculos que dizem respeito a ela mesma. Essa é a diferença entre uma entidade e uma estrutura de dados.

CLASSE	MÉTODO	O QUE RESOLVE
`Usuario`	`adicionarPontos`	Soma os pontos e dispara o recálculo do nível
`Usuario`	`pontosParaProximoNivel`	Quanto falta para subir de faixa
`MissaoSustentabilidade`	`concluir`	Gera a conclusão para um usuário
`MissaoSustentabilidade`	`estaValida`	Verifica se o prazo da missão faz sentido
`UsuarioMissao`	`concluidaNosUltimos`	Se a conclusão está dentro da janela de dias
`Loja`	`isAtiva`	Se a loja pode participar de resgate

B. Camadas da aplicação

Como as classes se organizam em camadas e quem depende de quem. As setas acompanham o sentido da requisição e nunca voltam: cada `Resource` depende apenas da BO correspondente, cada BO depende dos DAO de que precisa, e todos os DAO passam pela mesma `ConnectionFactory` — convergência que torna a troca de banco um problema de uma classe só.



7 Endpoints da API

Vinte e dois endpoints, sobre quatro recursos. A coluna da direita traz os códigos de status que cada um devolve no uso normal.

VERBO	URI	DESCRIÇÃO	STATUS
GET	`/usuarios`	Lista todos os usuários	`200`
POST	`/usuarios`	Cadastra um usuário	`201` · `400`
GET	`/usuarios/{id}`	Busca um usuário pelo id	`200` · `400` · `404`
PUT	`/usuarios/{id}`	Altera nome e e-mail	`200` · `400` · `404`
DELETE	`/usuarios/{id}`	Remove um usuário sem histórico	`204` · `400` · `404`
GET	`/usuarios/{id}/missoes`	Missões que o usuário concluiu	`200` · `404`
GET	`/usuarios/{id}/missoes/disponiveis`	Missões que ele ainda não concluiu	`200` · `404`
POST	`/usuarios/{id}/missoes`	Registra a conclusão de uma missão	`201` · `400` · `404` · `409`
GET	`/usuarios/{id}/resgates`	Cupons que o usuário resgatou	`200` · `404`
POST	`/usuarios/{id}/resgates`	Resgata um cupom numa loja	`201` · `400` · `403` · `404`
GET	`/lojas`	Lista as lojas; aceita filtro de ativas	`200`
POST	`/lojas`	Cadastra uma loja	`201` · `400`
GET	`/lojas/proximas`	Lojas ativas dentro do raio informado	`200` · `400`
GET	`/lojas/{id}`	Busca uma loja pelo id	`200` · `400` · `404`
PUT	`/lojas/{id}`	Altera uma loja	`200` · `400` · `404`
DELETE	`/lojas/{id}`	Remove uma loja sem resgates	`204` · `400` · `404`
GET	`/missoes`	Lista as missões	`200`
POST	`/missoes`	Cadastra uma missão	`201` · `400`
GET	`/missoes/{id}`	Busca uma missão pelo id	`200` · `400` · `404`
PUT	`/missoes/{id}`	Altera uma missão	`200` · `400` · `404`
DELETE	`/missoes/{id}`	Remove uma missão sem conclusões	`204` · `400` · `404`
GET	`/ranking`	Usuários ordenados por pontuação	`200`

Qualquer endpoint pode ainda devolver `500` em caso de falha de acesso ao banco. Em todo `POST` e `PUT`, o cabeçalho `Content-Type: application/json` é obrigatório.

8 Regras de negócio

Liberação do cupom

O resgate exige que **as duas** condições sejam verdadeiras ao mesmo tempo:

- o usuário estar a menos de **200 metros** da loja, pela distância calculada em Haversine;
- ter concluído **alguma missão nos últimos 7 dias**.

Falhando qualquer uma, a resposta é `403` e a mensagem diz qual das duas falhou. Não se trata de erro de permissão nem de autenticação: é regra de negócio, e o texto é escrito para ser mostrado ao usuário final.

Níveis de engajamento

PONTUAÇÃO	NÍVEL
de 0 a 499	Iniciante
de 500 a 799	Ecologista
de 800 a 999	Sustentavel Avancado
1000 ou mais	Lenda Verde

O nível é recalculado sozinho sempre que o usuário ganha pontos, dentro do próprio método `adicionarPontos` da entidade.

Conclusão de missão

A mesma pessoa não conclui a mesma missão duas vezes. A segunda tentativa devolve `409`, e a garantia final está na chave única do banco.

Exclusão com histórico

Usuário com missão concluída ou cupom resgatado não pode ser excluído, assim como loja com resgate e missão com conclusão. Apagar levaria junto o histórico que aponta para o registro. A BO verifica antes e responde `400` com a explicação, em vez de deixar o banco recusar e devolver um erro genérico.

9 Tratamento de erros

Todas as exceções passam por um único `@RestControllerAdvice`, que traduz cada tipo em um status HTTP e devolve sempre o mesmo formato de corpo. Do lado de quem consome, um tratador resolve a tela inteira.

```
{
  "erro": "Resgate bloqueado",
  "detalhe": "Voce ainda nao chegou na area da oferta."
}
```

STATUS	QUANDO ACONTECE
200	Consulta ou alteração concluída
201	Recurso criado
204	Remoção concluída, sem corpo de resposta
400	Campo inválido, id não numérico ou regra de negócio violada
403	Resgate bloqueado por distância ou por falta de missão recente
404	O id informado não existe
409	Missão já concluída por esse usuário
500	Falha de acesso ao banco

Validação de entrada

Os DTO de entrada usam Bean Validation. Quando há mais de um problema no mesmo envio, as mensagens vêm concatenadas, para que o formulário possa apontar todos de uma vez.

```
{
  "erro": "Dados invalidos",
  "detalhe": "E-mail invalido; O nome deve ter entre 3 e 80 caracteres;"
}
```

Mensagem de erro de banco

A mensagem original do Oracle é registrada no log do servidor, mas não vai para a resposta. Erros de banco carregam nomes de tabela, de coluna e de constraint, e isso não deve sair pela API. O cliente recebe uma mensagem genérica.

10 Deploy em nuvem

A aplicação está publicada e respondendo em endereço público.

PLATAFORMA	EMPACOTAMENTO	BANCO
Render	**Docker multi-stage**	**Oracle FIAP**

```
https://spotmydeal-sprint4.onrender.com
```

Imagem Docker

O `Dockerfile` tem dois estágios. O primeiro compila com Maven e é descartado inteiro; o segundo recebe apenas o `.jar` e roda sobre uma imagem com JRE 21. O resultado é uma imagem final sem Maven, sem código-fonte e sem as dependências de compilação.

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Credenciais

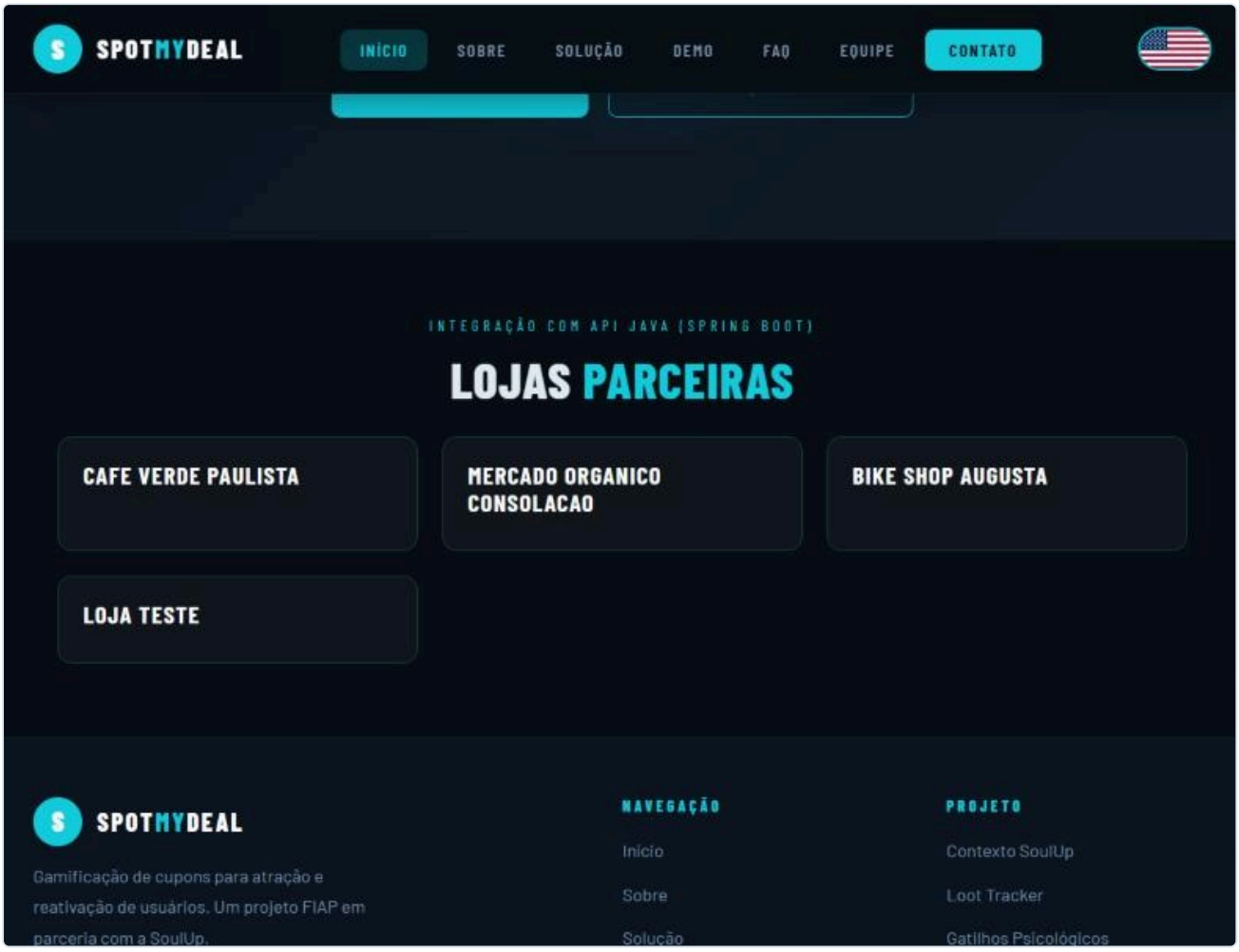
Nenhuma senha está no código. O `application.properties` lê usuário e senha de variável de ambiente, cadastradas no painel do provedor, e a porta vem de `PORT`, que o próprio Render injeta.

```
spring.datasource.url=${DB_URL:jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl}
spring.datasource.username=${DB_USER:rm570539}
spring.datasource.password=${DB_PASS:}
server.port=${PORT:8080}
```

A instância é do plano gratuito e hiberna após 15 minutos sem acesso, de modo que a primeira requisição depois desse intervalo pode levar até 50 segundos até a aplicação responder.

11 Integração com o front-end

A interface web, feita em React e publicada na Vercel, consome esta API. A seção **Lojas Parceiras** da página inicial é montada com o retorno de `GET /lojas`.



Página inicial do SpotMyDeal listando as lojas vindas do Oracle através da API

As quatro lojas exibidas são exatamente as quatro linhas da tabela `T_SMD_LOJA`, incluindo a que foi cadastrada por requisição `POST` durante os testes — o que mostra que a tela reflete o banco, e não uma lista fixa no código.

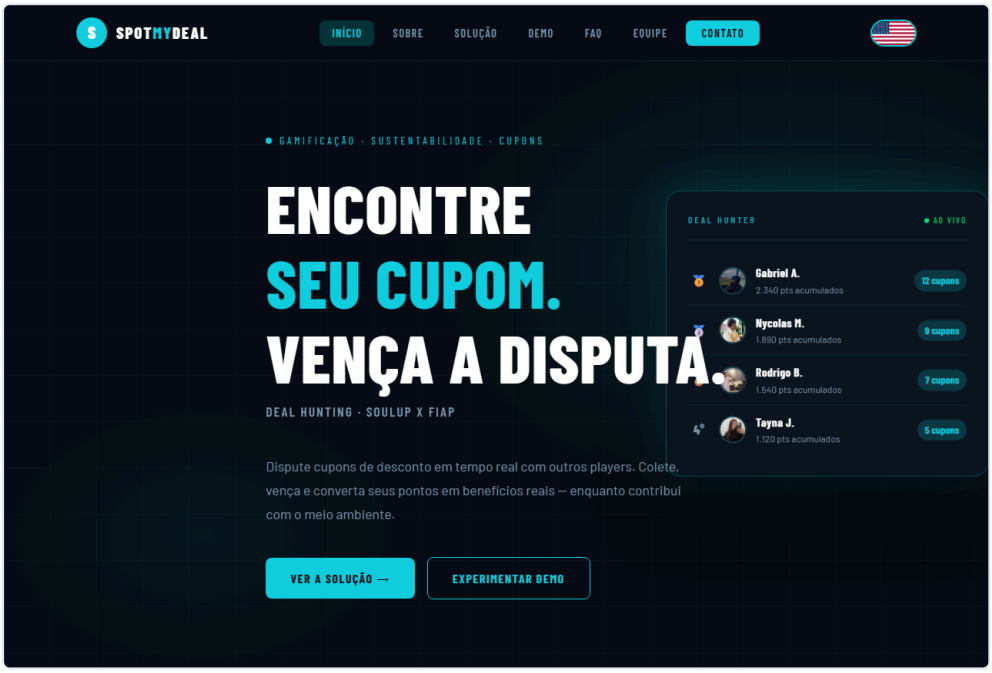
CORS

Como o front está num domínio diferente do da API, o navegador exigiria autorização explícita para a chamada. A configuração de CORS libera as origens para os métodos GET, POST, PUT, DELETE e OPTIONS; sem ela, a requisição seria barrada antes mesmo de chegar ao servidor.

12 Protótipo — telas do sistema

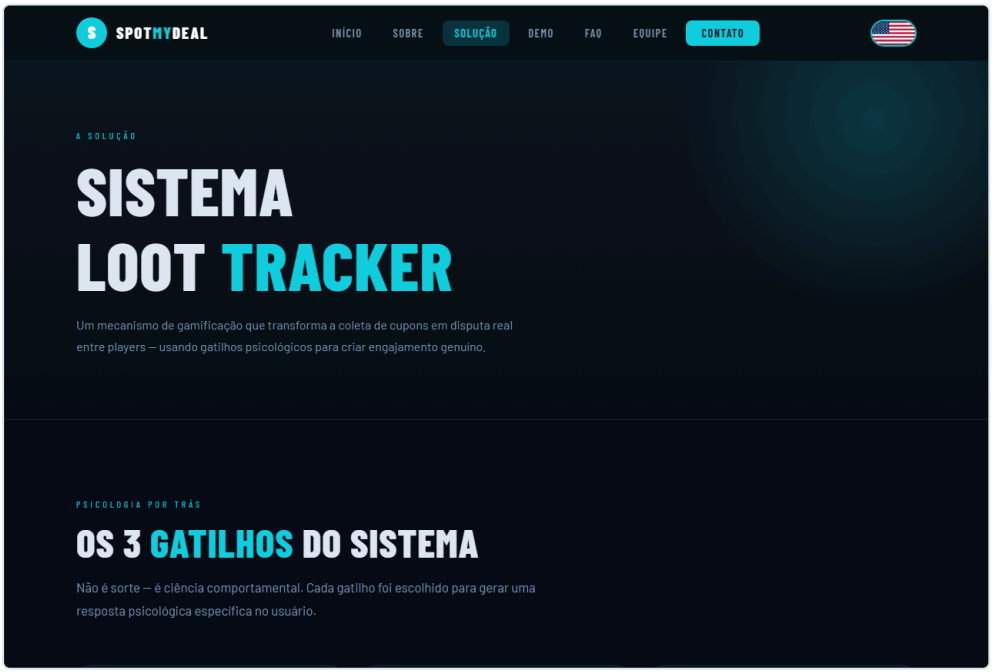
Capturas reais da aplicação publicada, feitas no navegador. A interface está em React e consome a API Java descrita neste documento.

Página inicial



Abertura do site, com a proposta da solução e o acesso às demais páginas

Solução



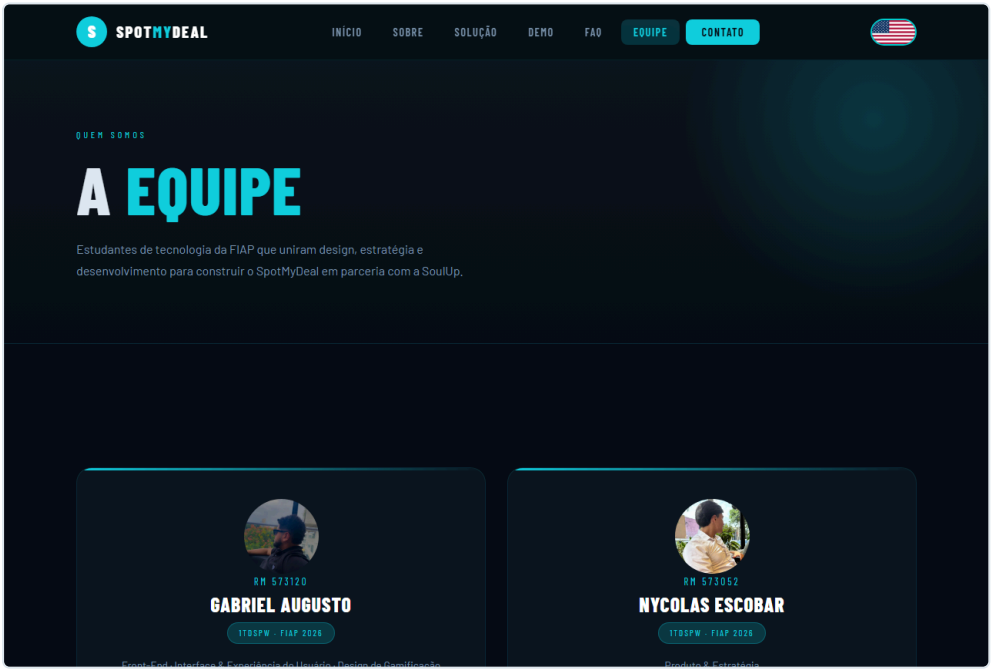
Apresentação do problema atacado e de como a plataforma resolve

Demonstração interativa



Simulação do mecanismo de disputa de cupons e acúmulo de pontos

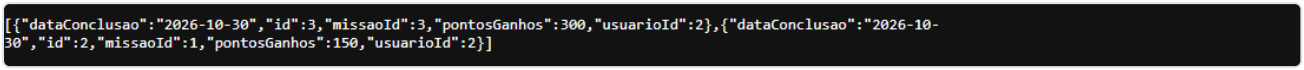
Equipe



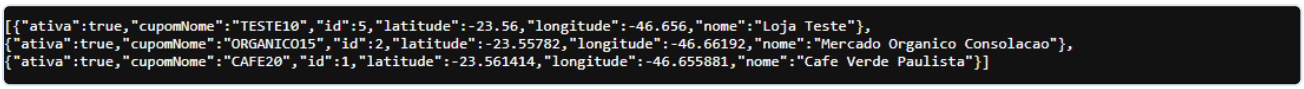
Integrantes do grupo, com RM e turma

Respostas da API

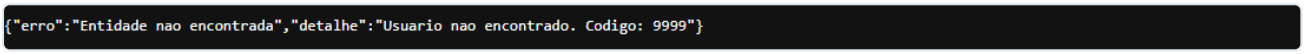
As três capturas abaixo vêm direto da aplicação publicada, abertas no navegador.



`GET /usuarios/2/missoes` — histórico de conclusões, lido da tabela associativa `T_SMD_USUARIO_MISSAO`



`GET /lojas/proximas?lat=-23.5596&lon=-46.6589&raio=500` — lojas ativas dentro do raio, da mais perto para a mais longe



`GET /usuarios/9999` — resposta 404, no mesmo formato de erro de toda a API

Sobre o tempo de resposta. A instância do Render é do plano gratuito e hiberna após 15 minutos sem acesso. A primeira chamada depois desse intervalo pode levar até 50 segundos; a partir da segunda, a resposta volta em menos de um segundo.

13 Procedimentos para execução

1. Criar a estrutura do banco

Abrir o `script.sql` no SQL Developer, conectado ao usuário Oracle da FIAP, e executar o arquivo inteiro. Ele cria cinco tabelas, cinco sequences, as chaves estrangeiras e as restrições, e insere os dados de exemplo. Ao final, executa consultas de conferência.

Para recomeçar do zero, os comandos de `drop` estão comentados no topo do arquivo, já na ordem correta: as tabelas filhas primeiro, senão o Oracle recusa a exclusão por causa das chaves estrangeiras.

2. Informar as credenciais

```
set DB_USER=seu_rm
set DB_PASS=sua_senha_do_oracle
```

3. Subir a aplicação

```
mvnw spring-boot:run
```

A API responde em `http://localhost:8080`. Para conferir que subiu, basta chamar `GET /usuarios`: a resposta deve ser `200` com a lista de usuários.

4. Testar os endpoints

O repositório acompanha uma coleção do Insomnia com as requisições prontas, organizadas por assunto e com corpo de exemplo em cada POST e PUT. A importação é feita em **File** → **Import**, e o ambiente já aponta para a URL de produção.

> **Observação sobre a raiz.** O endereço `/` não tem rota mapeada e responde `404`, o que é o comportamento esperado de uma API Restful: não existe página inicial, apenas os recursos. Para verificar que a aplicação está no ar, deve-se chamar `/usuarios`.

Ferramentas e versões

ITEM	VERSÃO / OBSERVAÇÃO
JDK	Java 21 — o projeto compila com `<java.version>21</java.version>`
IDE	IntelliJ IDEA; também abre em Eclipse ou VS Code
Maven	3.9.11 — o wrapper acompanha o projeto, não precisa instalar
Spring Boot	4.1.2
Banco	Oracle da FIAP, driver `ojdbc17`
SQL Developer	Para executar o `script.sql`
Insomnia	Para testar os endpoints

Como importar o projeto

O projeto é Maven, então a importação é direta em qualquer das IDE acima:

- **IntelliJ IDEA:** *File → Open*, apontando para a pasta do projeto. A IDE reconhece o `pom.xml` e baixa as dependências sozinha.
- **Eclipse:** *File → Import → Existing Maven Projects*, apontando para a pasta que contém o `pom.xml`.
- **VS Code:** abrir a pasta com a extensão *Extension Pack for Java* instalada.

Ao abrir o `.zip` da entrega, descompacte antes de importar: a pasta `soulup-api` que está dentro dele é a raiz do projeto.

14 Repositórios e links

API Java — esta entrega

ITEM	ENDEREÇO
Código-fonte	`github.com/RodrigoBanharelli/spotmydeal-sprint4`
Aplicação publicada	`spotmydeal-sprint4.onrender.com`
Verificação rápida	`spotmydeal-sprint4.onrender.com/usuarios`

Front-end

ITEM	ENDEREÇO
Código-fonte	`github.com/GRibeiro-developer/Spotmydeal-REACT`
Aplicação publicada	`spotmydeal-react-790tl8nny-gabriel-augusto1.vercel.app`

O que está no repositório da API

CAMINHO	CONTEÚDO
`src/main/java`	As sete camadas, em 39 classes
`script.sql`	Criação das tabelas, sequences, constraints e dados de exemplo
`Dockerfile`	Imagem multi-stage usada no deploy
`README.md`	Arquitetura, endpoints e instruções de execução
`entrega/`	O projeto compactado em `.zip` e esta documentação em PDF, conforme exigido na entrega
`docs/`	Fontes dos diagramas usados nesta documentação

Equipe NextStage. Tayna Jimenes Silva (RM 569337), Gabriel Augusto (RM 573120), Rodrigo Banharelli de Souza (RM 570539) e Nycolas Melo Escobar (RM 573052) — 1TDSPW, FIAP, 2º semestre de 2026.